



Java For Testers

Fecha: 01/09/2025

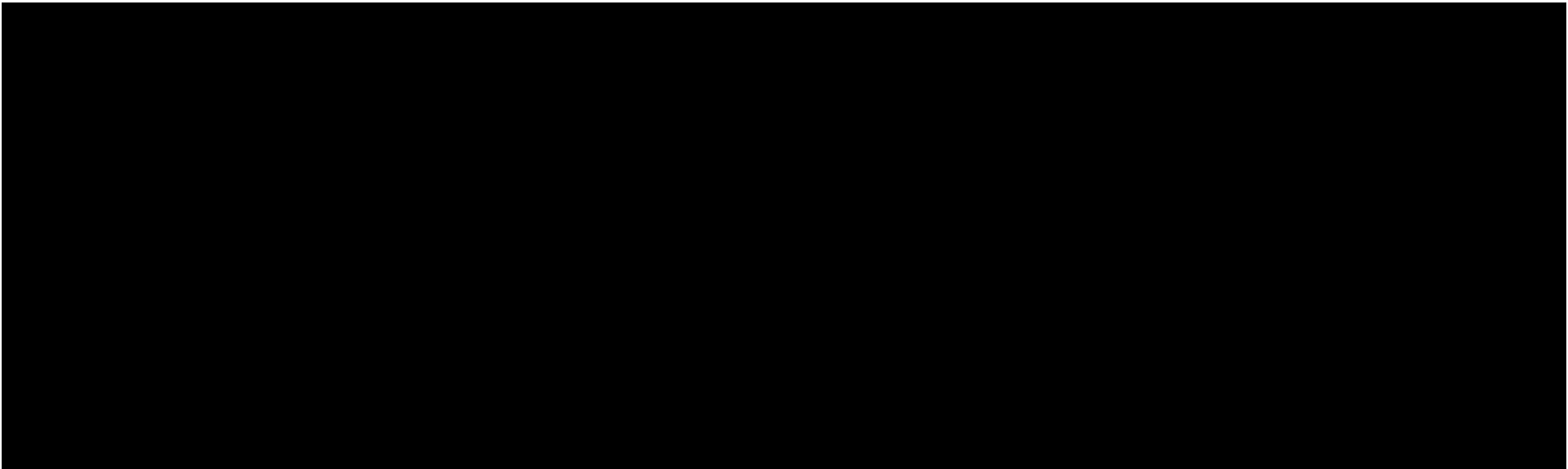
5

Métodos y Alcance

1. Métodos
 - void
 - return type
 - Parámetros
2. Modificadores
 - Private
 - Public
 - Protected
3. Invocar métodos desde otra clase
 - Static
 - Instancia de Clase

5. Métodos y alcance

5.1 Métodos



Método de programación

- ★ Es un bloque de código que realiza una tarea específica.
- ★ Permite reutilizar código sin tener que escribirlo varias veces.
- ★ Puede recibir datos (parámetros) y regresar un resultado (return).
- ★ Ayuda a organizar mejor los programas.

¿Por qué un Tester debería saberlo?

- ★ Para validar reglas de negocio.
- ★ Porque los frameworks de automatización están llenos de métodos.
- ★ Para reutilizar acciones comunes como:
 - Login
 - Tomar screenshot
 - Dar click
 - Leer datos de prueba
- ★ Hace que las pruebas sean más fáciles de mantener.
- ★ Reduce código duplicado.

Método VOID

- ★ Es un método que no regresa ningún valor.
- ★ Se utiliza cuando solo necesitas ejecutar una acción.
- ★ Puede recibir parámetros o no recibir ninguno.



Recuerda

- ★ **void** = no regresa nada.
- ★ Ejecuta una tarea y termina.

```
// Método void sin parámetros
public static void saludar() {
    System.out.println("Hola Java For Testers");
}

// Método void con parámetro "nombre"
public static void saludar(String nombre) {
    System.out.println("Hola " + nombre);
}
```



Método Return Type

- ★ Es un método que no regresa ningún valor.
- ★ Son métodos que regresan un valor al terminar su ejecución.
- ★ El tipo de dato que regresan se define antes del nombre del método.
- ★ Se utiliza la palabra reservada `return`.



Recuerda

- ★ Un método `void` ejecuta una acción.
- ★ Un método con `return` ejecuta una acción y además entrega un resultado.

```
// Método que regresa el texto "Bicho"
public static String obtenerNombre() {
    return "Bicho";
}

// Método que regresa un número resultado de
// sumar 2 números (parámetros)
public static int sumar(int num1, int num2) {
    return num1 + num2;
}
```



Parámetros

- ★ Son **variables que recibe un método** para trabajar con datos externos.
- ★ Permiten **reutilizar un mismo método** con diferentes valores.
- ★ Se definen entre los paréntesis **()** del método.
- ★ Pueden ser usados en métodos **void** o **return**.



Recuerda

- ★ Los parámetros se definen en **()**.
- ★ Puedes tener cero, uno o varios parámetros.
- ★ Permiten reutilizar métodos con diferentes datos.

```
public static void saludar(String nombre) {  
    System.out.println("Hola " + nombre);  
}
```



Actividad

Realiza la siguiente actividad



1. Ve al código de ejemplo del repositorio
2. Analiza el código de condicionales en el paquete “src/test/java/cap05/metodos”
3. Importa el código a tu computador, ya hemos visto opciones de esto en capítulos anteriores
4. Entiende el código, ejecutalo

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

Actividad

Realiza la siguiente actividad

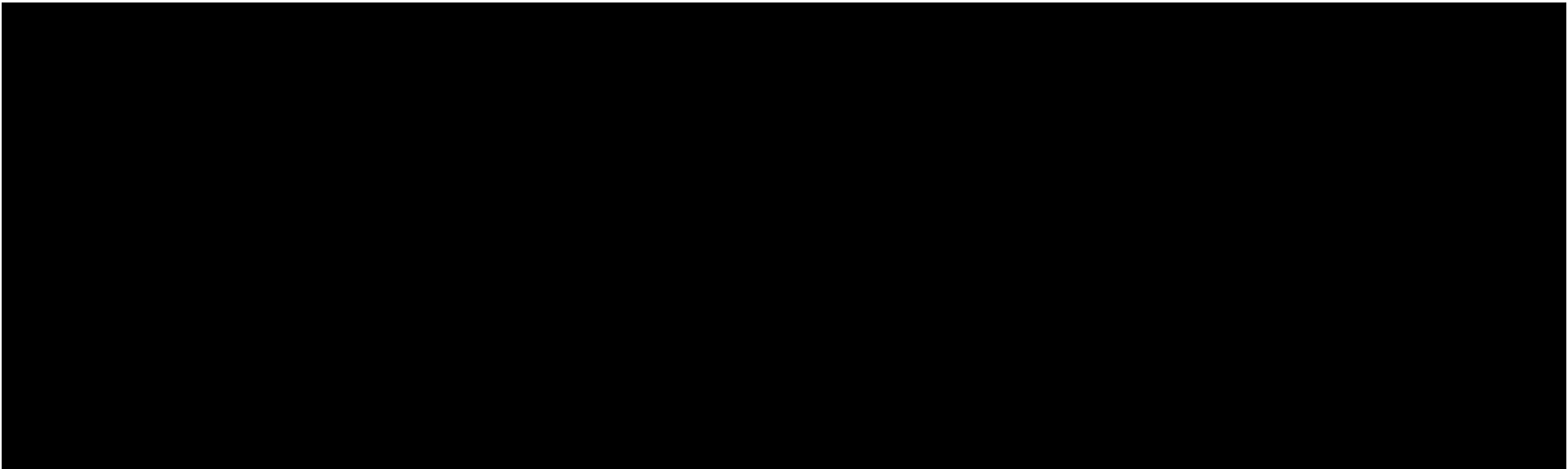


1. Abre IntelliJ
2. Comienza a crear códigos de ejemplo relacionados a crear métodos en el proyecto, package y archivo Java de tu elección

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

5.2 Modificadores



¿Qué son los Modificadores?

- ★ Son palabras clave que definen **desde dónde se puede acceder** a un método.
- ★ Ayudan a controlar la **visibilidad** y **protección** del código.
- ★ Permiten decidir quién puede usar un método y quién no.

¿Por qué un Tester debería saber usar FOR?

- ★ Porque los encontrarás constantemente en frameworks de automatización.
- ★ Para entender qué métodos puedes invocar desde tus pruebas.
- ★ Para crear frameworks más organizados y mantenibles.
- ★ Para leer código de desarrolladores con mayor facilidad.

Private

- ★ Es el modificador de acceso **más restrictivo**.
- ★ El método solo puede ser utilizado **dentro de la misma clase**.
- ★ **No** puede ser utilizado desde **otras clases**.
- ★ **No** puede ser utilizado desde **otros packages**.
- ★ Se usa para **ocultar lógica interna** que no debe ser accesible desde el exterior.
- ★ Llevan la palabra **private** al inicio del método (antes del tipo y nombre).
- ★ Pueden ser **void** o **return**.

Sí **public** es una puerta abierta para todos, **private** es una puerta que solo puede usar la propia clase.

```
private void validarPassword() {  
    System.out.println("Validar password...");  
}
```



Public

- ★ Es el modificador de acceso más abierto.
- ★ El método puede ser utilizado desde **cualquier clase**.
- ★ El método puede ser utilizado desde **cualquier package**.
- ★ Llevan la palabra **public** al inicio del método (antes del tipo y nombre)
- ★ Pueden ser **void** o **return**

public = accesible desde cualquier clase y cualquier package.

```
public void login() {  
    System.out.println("Login exitoso");  
}
```



Protected

- ★ Es un modificador de **acceso intermedio**.
- ★ El método puede ser utilizado dentro de la **misma clase**.
- ★ El método puede ser utilizado desde **otras clases** del **mismo package**.
- ★ **No** puede ser utilizado desde **clases ubicadas en otros packages**.
- ★ Se usa cuando queremos **permitir reutilización**, pero sin exponer completamente el método.
- ★ Llevan la palabra **protected** al inicio del método (antes del tipo y nombre).
- ★ Pueden ser **void** o **return**.

```
protected void abrirNavegador() {  
    System.out.println("Abriendo navegador...");  
}
```

- ★ **private** → solo la misma clase puede acceder.
- ★ **public** → todos pueden acceder.
- ★ **protected** → la familia puede acceder (misma clase, mismo package y clases hijas).



Actividad

Realiza la siguiente actividad



1. Ve al código de ejemplo del repositorio
2. Analiza el código de condicionales en el paquete
“src/test/java/cap05/modificadores”
3. Importa el código a tu computador, ya hemos visto opciones de esto en capítulos anteriores
4. Entiende el código, ejecutalo

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

Actividad

Realiza la siguiente actividad

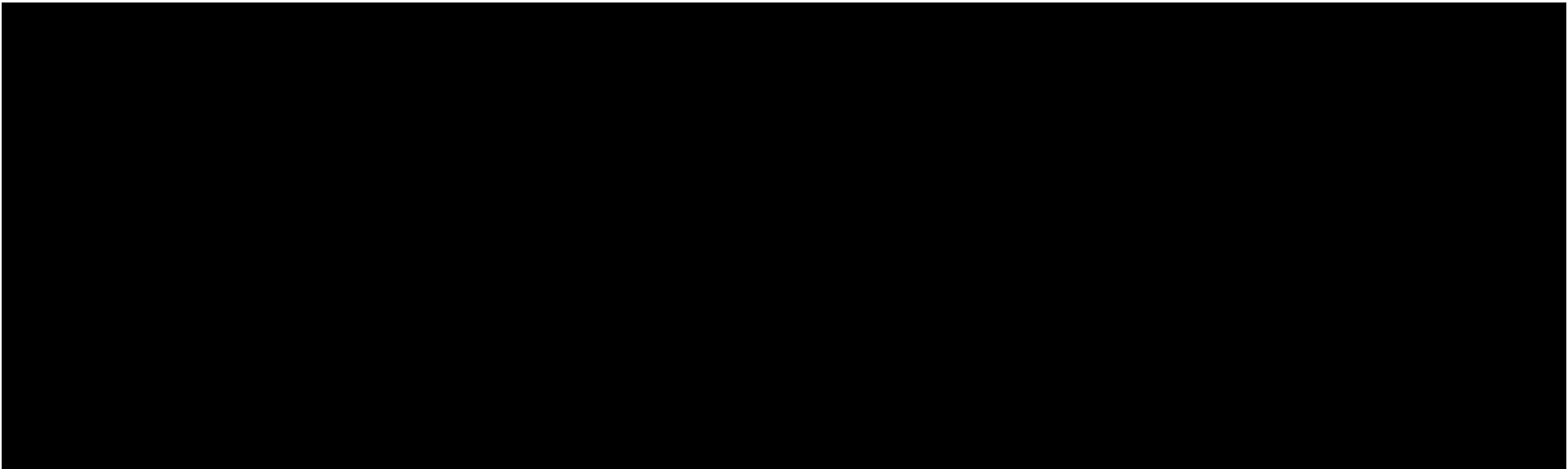


1. Abre IntelliJ
2. Comienza a crear códigos de ejemplo relacionados a crear métodos con algún tipo de modificador en el proyecto, package y archivo Java de tu elección

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

5.3 Invocar métodos desde otra clase



Clase Origen

```
public class Utilidades {  
  
    public static void saludar() {  
        System.out.println( "Hola Java For Testers" );  
    }  
  
}
```

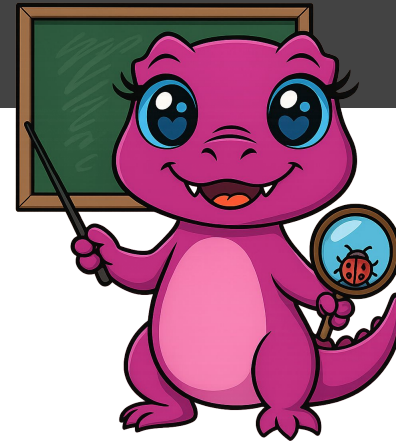
Clase Destino

```
public class JavaForTesters {  
  
    public static void main(String[] args) {  
        Utilidades.saludar();  
    }  
  
}
```



Recuerda

- ★ El Método de la clase origen debe ser **static**.
- ★ Usa el nombre de la clase.
- ★ Sintaxis: **Clase.metodo()**;



Clase Origen

```
public class Utilidades {  
  
    public void saludar() {  
        System.out.println( "Hola Java For Testers" );  
    }  
  
}
```

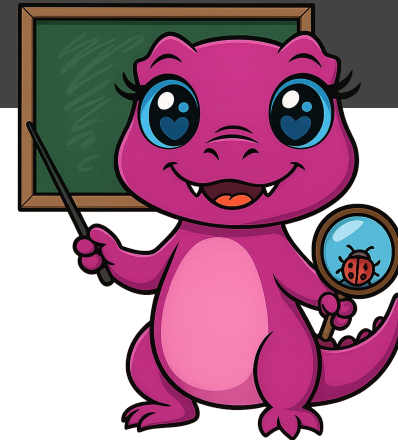
Clase Destino

```
public class JavaForTesters {  
  
    public static void main(String[] args) {  
        Utilidades util = new Utilidades();  
        util.saludar();  
    }  
  
}
```



Recuerda

- ★ El método de la clase origen **NO debe ser static**.
- ★ Debes crear una instancia de la clase.
- ★ Sintaxis: **Clase objeto = new Clase();**
- ★ Invoca el método usando el objeto.



Actividad

Realiza la siguiente actividad



1. Ve al código de ejemplo del repositorio
2. Analiza el código de condicionales en el paquete
“src/test/java/cap05/invocarMetodos”
3. Importa el código a tu computador, ya hemos visto opciones de esto en capítulos anteriores
4. Entiende el código, ejecutalo

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

Actividad

Realiza la siguiente actividad



1. Abre IntelliJ
2. Comienza a crear códigos de ejemplo relacionados a crear métodos con algún tipo de modificador y que se puedan invocar desde otra clase o la misma según su modificador, en el proyecto, package y archivo Java de tu elección

****NOTA**

Los nombres de packages, directorios y archivos pueden cambiar en función de lo que el alumno elija no es forzoso mantener los nombre recomendados

Quizz

Quizz

Realiza los siguientes

- ★ [Quiz 01](#)
- ★ [Quiz 02](#)
- ★ [Quiz 03](#)

Recuerda:

- ★ Trata de no memorizar orden de las respuestas, entiende, asimila y aprende
- ★ En la página del curso, todos los capítulos tienen su acceso directo a los quizzes



Recursos del curso

Recursos

En esta sección encontrarás
recursos más importantes del
curso

- ★ [Página del curso](#)
- ★ [Repositorio de Git](#)
- ★ [YouTube videos del curso](#)



Si deseas cursos personalizados u
otro servicio no dudes en
contactarnos

<https://testeracademy.com.mx/contacto>

Gracias

 “No aprendemos para aprobar pruebas, aprendemos para transformar realidades.”



¡Hasta pronto!